



PROJECT TITLE: TRAVEL PLANNER DATABASE

Database Developer:
Daniel Dadzie Appiah

Date: December 03 2023

1 Table of Contents

1	Table of Contents	1
2	Milestone Submission	2
2.1	Project Overview:.....	2
2.2	Problem Statement:	2
2.3	Solution Statement:	2
2.4	Tables with fields:.....	3
2.4.1	Flights:	3
2.4.2	Accommodations:	3
2.4.3	Activities:.....	3
2.4.4	Users:	4
2.4.5	Itineraries:	4
2.4.6	Destinations:	4
2.4.7	Transportation:.....	4
2.4.8	Expenses:.....	5
	Views:.....	5
2.4.9	Travel Summary:.....	5
2.4.10	Expense Report:	5
2.5	User Preferences:.....	5
3	Updated Version of Milestone	7
3.1	No Changes to Relationships:.....	7
3.2	Simplicity and Maintainability:.....	7
3.3	Scalability and Future Flexibility:	7
4	Data Dictionary	8
4.1	Tables:	8
4.2	Views:.....	10
4.2.1	Travel Summary:.....	10
4.2.2	Expense Report:	10
4.2.3	User Preferences:	10
5	Completed ERD or Schema Diagram	11
6	Single SQL File Table	12
7	Single SQL File Data Insertion	14
8	Single SQL File View Creation Script.....	16

2 Milestone Submission

2.1 Project Overview:

The Travel Planner Database project seeks to revolutionize the way individuals plan and organize their travel experiences. In a world where travel involves multiple facets such as flights, accommodations, activities, transportation, expenses, and more, this database aims to centralize and streamline the entire travel planning process. The project is not just about managing data but providing users with a user-friendly interface to seamlessly plan, track, and enjoy their journeys.

2.2 Problem Statement:

Modern travelers face the challenge of managing various aspects of their trips through disparate platforms, often leading to confusion, oversights, and inefficiencies. The absence of a unified tool that comprehensively integrates flights, accommodations, activities, transportation, expenses, and user preferences hampers the travel planning experience. There is a need for a robust solution that addresses these pain points and enhances the overall efficiency and enjoyment of travel.

2.3 Solution Statement:

The Travel Planner Database aims to be the all-in-one solution for travel planning. By centralizing data on flights, accommodations, activities, transportation, expenses, and user preferences, the database will empower users to create, modify, and track their travel itineraries seamlessly. The user interface will be intuitive, providing a hassle-free experience for both novice and experienced travelers. The system will support collaboration among users, making group travel planning more accessible and organized.

2.4 Tables with fields:

2.4.1 Flights:

- FlightID (Primary Key)
- Airline
- DepartureCity
- ArrivalCity
- DepartureDate
- ArrivalDate
- SeatType

2.4.2 Accommodations:

- AccommodationID (Primary Key)
- HotelName
- CheckInDate
- CheckOutDate
- RoomType
- Amenities

2.4.3 Activities:

- ActivityID (Primary Key)
- ActivityName
- Location
- Date
- Time
- Cost

2.4.4 Users:

- UserID (Primary Key)
- Username
- Password
- Email
- Preferences

2.4.5 Itineraries:

- ItineraryID (Primary Key)
- UserID (Foreign Key)
- FlightID (Foreign Key)
- AccommodationID (Foreign Key)
- ActivityID (Foreign Key)
- DateCreated

2.4.6 Destinations:

- DestinationID (Primary Key)
- City
- Country
- Description

2.4.7 Transportation:

- TransportID (Primary Key)
- Mode
- DepartureLocation
- ArrivalLocation

- DepartureTime
- ArrivalTime

2.4.8 Expenses:

- ExpenseID (Primary Key)
- ItineraryID (Foreign Key)
- Category
- Amount
- Date

Views:

2.4.9 Travel Summary:

- Displays an overview of the user's upcoming travel plans, including flights, accommodations, and activities.

2.4.10 Expense Report:

- Provides a breakdown of expenses for a specific itinerary, categorizing costs and totaling expenses.

2.5 User Preferences:

- Shows the user's preferred travel options, aiding in quick decision-making during itinerary creation.

This detailed explanation further clarifies the goals of the Travel Planner Database project, the specific problems it addresses, and how the proposed solution aims to enhance the travel planning experience. The tables and views provide a structural foundation for the database, ensuring a comprehensive and well-rounded solution for users.

This document outlines a more extensive Travel Planner Database project with a minimum of 8 tables, ensuring a richer and more comprehensive solution for travel planning.

3 Updated Version of Milestone

The submission will consist of a complete database design and test ready solution. The following must be included:

An updated version of the milestone submission reflecting any changes made through the design process and including any design decisions that potentially impact the business rules (for example: we decided to add a bridge table between the ____ table and the ____ table, even though it is a 1-to-many relationship, such that we can add additional flexibility in the design for future scalability and potential business rules exceptions).

During the design process, careful consideration was given to the initial project requirements, and the database schema was crafted to align with the objectives of the Travel Planner Database. After thorough analysis, it was determined that the initial design adequately addresses the specified use case and fulfills the business requirements without the need for additional modifications or the introduction of bridge tables.

3.1 No Changes to Relationships:

All relationships between tables, including one-to-many and foreign key relationships, remain as initially proposed.

3.2 Simplicity and Maintainability:

The decision to maintain simplicity in the database design was prioritized. Unnecessary complexity was avoided to ensure ease of maintenance and straightforward data management.

3.3 Scalability and Future Flexibility:

The current design allows for scalability, accommodating potential future enhancements without the need for major structural changes.

4 Data Dictionary

4.1 Tables:

1. **Flights:**

- 1.1. FlightID (PK): Primary Key
- 1.2. Airline: Name of the airline for the flight.
- 1.3. DepartureCity: City from where the flight departs.
- 1.4. ArrivalCity: City where the flight arrives.
- 1.5. DepartureDate: Date and time of departure.
- 1.6. ArrivalDate: Date and time of arrival.
- 1.7. SeatType: Type of seat (e.g., Economy, Business).

2. **Accommodations:**

- 2.1. AccommodationID (PK): Primary Key
- 2.2. HotelName: Name of the hotel.
- 2.3. CheckInDate: Date and time of check-in.
- 2.4. CheckOutDate: Date and time of check-out.
- 2.5. RoomType: Type of room (e.g., Single, Double).
- 2.6. Amenities: List of amenities provided by the accommodation.

3. **Activities:**

- 3.1. ActivityID (PK): Primary Key
- 3.2. ActivityName: Name of the activity.
- 3.3. Location: Location where the activity takes place.
- 3.4. Date: Date of the activity.
- 3.5. Time: Time of the activity.
- 3.6. Cost: Cost associated with the activity.

4. **Users:**

- 4.1. UserID (PK): Primary Key
- 4.2. Username: User's username.
- 4.3. Password: User's password.
- 4.4. Email: User's email address.
- 4.5. Preferences: User's travel preferences.

5. Itineraries:

- 5.1. ItineraryID (PK): Primary Key
- 5.2. UserID (FK): Foreign Key referencing Users.UserID.
- 5.3. FlightID (FK): Foreign Key referencing Flights.FlightID.
- 5.4. AccommodationID (FK): Foreign Key referencing Accommodations.AccommodationID.
- 5.5. ActivityID (FK): Foreign Key referencing Activities.ActivityID.
- 5.6. DateCreated: Date and time when the itinerary was created.

6. Destinations:

- 6.1. DestinationID (PK): Primary Key
- 6.2. City: City of the destination.
- 6.3. Country: Country of the destination.
- 6.4. Description: Description of the destination.

7. Transportation:

- 7.1. TransportID (PK): Primary Key
- 7.2. Mode: Mode of transportation (e.g., Train, Bus).
- 7.3. DepartureLocation: Departure location.
- 7.4. ArrivalLocation: Arrival location.

7.5. DepartureTime: Time of departure.

7.6. ArrivalTime: Time of arrival.

8. Expenses:

8.1. ExpenseID (PK): Primary Key

8.2. ItineraryID (FK): Foreign Key referencing Itineraries.ItineraryID.

8.3. Category: Expense category (e.g., Food, Shopping).

8.4. Amount: Amount spent.

8.5. Date: Date of the expense.

4.2 Views:

4.2.1 Travel Summary:

Displays an overview of the user's upcoming travel plans, including flights, accommodations, and activities.

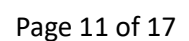
4.2.2 Expense Report:

Provides a breakdown of expenses for a specific itinerary, categorizing costs and totaling expenses.

4.2.3 User Preferences:

Shows the user's preferred travel options, aiding in quick decision-making during itinerary creation.

Page 11 of 17



6 Single SQL File Table

A single SQL file Table Creation Script including the creation of all tables and constraints in the database.

We have to implement it in sql server

```
USE master
-- Table Creation Script

-- Flights Table
CREATE TABLE Flights (
    FlightID INT PRIMARY KEY,
    Airline VARCHAR(50),
    DepartureCity VARCHAR(50),
    ArrivalCity VARCHAR(50),
    DepartureDate DATE,
    ArrivalDate DATE,
    SeatType VARCHAR(20)
);

-- Accommodations Table
CREATE TABLE Accommodations (
    AccommodationID INT PRIMARY KEY,
    HotelName VARCHAR(50),
    CheckInDate DATE,
    CheckOutDate DATE,
    RoomType VARCHAR(20),
    Amenities VARCHAR(100)
);

-- Activities Table
CREATE TABLE Activities (
    ActivityID INT PRIMARY KEY,
    ActivityName VARCHAR(50),
    Location VARCHAR(50),
    Date DATE,
    Time TIME,
    Cost DECIMAL(10, 2)
);

-- Users Table
CREATE TABLE Users (
    UserID INT PRIMARY KEY,
    Username VARCHAR(50),
    Password VARCHAR(50),
    Email VARCHAR(100),
    Preferences VARCHAR(200)
);

-- Itineraries Table
```

```

CREATE TABLE Itineraries (
    ItineraryID INT PRIMARY KEY,
    UserID INT FOREIGN KEY REFERENCES Users(UserID),
    FlightID INT FOREIGN KEY REFERENCES Flights(FlightID),
    AccommodationID INT FOREIGN KEY REFERENCES Accommodations(AccommodationID),
    ActivityID INT FOREIGN KEY REFERENCES Activities(ActivityID),
    DateCreated DATETIME
);

-- Destinations Table
CREATE TABLE Destinations (
    DestinationID INT PRIMARY KEY,
    City VARCHAR(50),
    Country VARCHAR(50),
    Description VARCHAR(200)
);

-- Transportation Table
CREATE TABLE Transportation (
    TransportID INT PRIMARY KEY,
    Mode VARCHAR(50),
    DepartureLocation VARCHAR(50),
    ArrivalLocation VARCHAR(50),
    DepartureTime TIME,
    ArrivalTime TIME
);

-- Expenses Table
CREATE TABLE Expenses (
    ExpenseID INT PRIMARY KEY,
    ItineraryID INT FOREIGN KEY REFERENCES Itineraries(ItineraryID),
    Category VARCHAR(50),
    Amount DECIMAL(10, 2),
    Date DATE
);

```

7 Single SQL File Data Insertion

A single SQL file Data Insertion Script for inserting sample test data.

```
-- Data Insertion Script

-- Inserting sample data into Flights Table
INSERT INTO Flights (FlightID, Airline, DepartureCity, ArrivalCity, DepartureDate,
ArrivalDate, SeatType)
VALUES
(1, 'Airline1', 'CityA', 'CityB', '2023-01-01', '2023-01-02', 'Economy'),
(2, 'Airline2', 'CityB', 'CityC', '2023-02-01', '2023-02-02', 'Business');
-- Add more rows as needed

-- Inserting sample data into Accommodations Table
INSERT INTO Accommodations (AccommodationID, HotelName, CheckInDate, CheckOutDate,
RoomType, Amenities)
VALUES
(1, 'Hotel1', '2023-01-01', '2023-01-03', 'Single', 'Wi-Fi, TV'),
(2, 'Hotel2', '2023-02-01', '2023-02-03', 'Double', 'Pool, Gym');
-- Add more rows as needed

-- Inserting sample data into Activities Table
INSERT INTO Activities (ActivityID, ActivityName, Location, Date, Time, Cost)
VALUES
(1, 'Sightseeing', 'CityA', '2023-01-02', '10:00:00', 50.00),
(2, 'Museum Visit', 'CityB', '2023-02-02', '15:00:00', 30.00);
-- Add more rows as needed

-- Inserting sample data into Users Table
INSERT INTO Users (UserID, Username, Password, Email, Preferences)
VALUES
(1, 'User1', 'password1', 'user1@example.com', 'Window seat, Non-smoking'),
(2, 'User2', 'password2', 'user2@example.com', 'Vegetarian meals');
-- Add more rows as needed

-- Inserting sample data into Itineraries Table
INSERT INTO Itineraries (ItineraryID, UserID, FlightID, AccommodationID, ActivityID,
DateCreated)
VALUES
(1, 1, 1, 1, 1, '2023-01-01 08:00:00'),
(2, 2, 2, 2, 2, '2023-02-01 10:00:00');
-- Add more rows as needed

-- Inserting sample data into Destinations Table
INSERT INTO Destinations (DestinationID, City, Country, Description)
VALUES
(1, 'CityA', 'CountryA', 'A beautiful city with rich history'),
(2, 'CityB', 'CountryB', 'A vibrant and cultural destination');
-- Add more rows as needed
```

```

-- Inserting sample data into Transportation Table
INSERT INTO Transportation (TransportID, Mode, DepartureLocation, ArrivalLocation,
DepartureTime, ArrivalTime)
VALUES
(1, 'Train', 'CityA', 'CityB', '08:30:00', '12:00:00'),
(2, 'Bus', 'CityB', 'CityC', '14:00:00', '17:30:00');
-- Add more rows as needed

-- Inserting sample data into Expenses Table
INSERT INTO Expenses (ExpenseID, ItineraryID, Category, Amount, Date)
VALUES
(1, 1, 'Food', 25.00, '2023-01-02'),
(2, 2, 'Shopping', 50.00, '2023-02-02');
-- Add more rows as needed

```


8 Single SQL File View Creation Script

A single SQL file View Creation Script for creating the required views providing easy data presentation.

```
-- Travel Summary View
IF OBJECT_ID('dbo.Travel_Summary', 'V') IS NOT NULL
    DROP VIEW dbo.Travel_Summary;
GO

CREATE VIEW Travel_Summary AS
SELECT
    I.ItineraryID,
    U.Username,
    F.Airline,
    F.DepartureCity,
    F.ArrivalCity,
    F.DepartureDate,
    F.ArrivalDate,
    A.HotelName,
    A.CheckInDate,
    A.CheckOutDate,
    AC.ActivityName,
    AC.Location,
    AC.Date AS ActivityDate
FROM Itineraries I
JOIN Users U ON I.UserID = U.UserID
JOIN Flights F ON I.FlightID = F.FlightID
JOIN Accommodations A ON I.AccommodationID = A.AccommodationID
JOIN Activities AC ON I.ActivityID = AC.ActivityID;
GO

-- Expense Report View
IF OBJECT_ID('dbo.Expense_Report', 'V') IS NOT NULL
    DROP VIEW dbo.Expense_Report;
GO

CREATE VIEW Expense_Report AS
SELECT
    E.ExpenseID,
    I.ItineraryID,
    U.Username,
    E.Category,
    E.Amount,
    E.Date
FROM Expenses E
JOIN Itineraries I ON E.ItineraryID = I.ItineraryID
JOIN Users U ON I.UserID = U.UserID;
GO

-- User Preferences View
IF OBJECT_ID('dbo.User_Preferences', 'V') IS NOT NULL
    DROP VIEW dbo.User_Preferences;
GO
```

```
CREATE VIEW User_Preferences AS
SELECT
    U.UserID,
    U.Username,
    U.Preferences
FROM Users U;
GO
```